Concurrencia y Paralelismo

Antonio Espín Herranz

Contenidos

- Visión práctica de std::thread, std::async y pools de hilos.
- Manejo seguro de recursos: RAII, mutexes y lock guards.
- Patrones comunes en servicios concurrentes.
- Introducción a Boost.Asio cuando aplique.

std:thread

Contenidos

- Clase thread
- Paso de parámetros a los hilos.
- Regiones críticas, interbloqueos, condiciones de carrera.
- Mecanismos de sincronización en hilos:
  - Mutex
- Variables de condición.
- Esquema productor / consumidor.
- Futures y tareas asíncronas.

threads

- Soporte en C++11
- Para trabajar con hilos, incluir el fichero .H
  - **#include <thread>**

- Para compilar con g++:
  - **g++ –std=c++11 fichero.cpp –o fichero –lpthread**

- Para compilar con make:
  - set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -lpthread")
  - set (CMAKE_CXX_STANDARD 11)
  - set (CMAKE_CXX_STANDARD_REQUIRED ON)

Lanzamiento de Hilos

- En C++11 un hilo se puede lanzar de 3 formas distintas:
  - Con una función.
    - La función puede tener parámetros o no.
  - Con un objeto de una clase que implemente el operador ()
    - También puede ser una estructura con la implementación de dicho operador.
  - Con una función lambda.

Con una función

- Primero se define una función:

```
void funcion_hello(){
        int i;
        for (i = 0 ; i < 10 ; i++)
                std::cout << "Hello " << i << std::endl;
}

std::thread h1(funcion_hello);
h1.join();
```

- También se puede inicializar el hilo con las {}

  std::thread h2 {función_hello}

Con una clase + operador ()

```
class MiFuncion {
        public:
                void operator()(){
                        for (int i = 0 ; i < 10 ; i++)
                                std::cout << "Operador () " << i << std::endl;
                }
};

std::thread h2( (MiFuncion()) );
h2.join();

// Ojo, se instancia la clase MiFuncion() se necesitan los paréntesis extras.
```

Con una función lambda

```
std::thread h3([]{
        for (int i = 0 ; i < 10 ; i++)
                std::cout << "Lambda " << i << std::endl;
        });

h3.join();
```

Condiciones de carrera

```
int x = 42;
void f () { ++x; }
void g() { x=0; }
void h() { cout << "Hola" << endl; }
void i () { cout << "Adios" << endl; }

// La variable x las comparten dos hilos sin ningún tipo de protección.
void carrera() {
    thread t1{ f };
    thread t2{g};
    t1.join () ;
    t2.join () ;
    thread t3{h};
    thread t4{ i };
    t3.join ();
    t4.join () ;
}
```

Paso de argumentos a un hilo

- A un hilo se le pueden pasar un número indeterminado de argumentos.
- La función que ejecute el hilo tiene que tener todos esos argumentos.
- Al instanciar el hilo se le manda como primer parámetro la función que tiene que ejecutar.

```
void funcion(int x, std::string s){
    std::cout << "Parametro int: " << x << std::endl;
    std::cout << "Parametro string: " << s << std::endl;
}

// suele hacer un casting automático de const char * a std::string
std::thread hilo(funcion, 1, std::string("hola"));
hilo.join()
```

Paso de argumentos a un hilo

- La definición de la clase thread:
- El constructor recibe una función y un número indeterminado de argumentos, que pueden ser 0 o n
- **thread thread(Function&& *f*, Args&&... *args*);**

- Un hilo termina cuando finaliza la rutina que ejecuta (por ejemplo, realiza un proceso n veces y termina) y llama a la instrucción **return**.

Paso de parámetros por referencia

- Cuando queremos pasar un parámetro a un hilo por referencia se tiene que indicar en la construcción del hilo.
- Para ello se dispone de la función std::ref(param)

- #include <functional>
- #include <thread>

- **void** f (registro & r) ;

- **void** g(registro & s) {
  - thread t1{ f ,s}; // Copia de s
  - thread t2{ f , std::ref (s) }; // Referencia a s
  - Thread t3 {[&] { f (s) ; }}; // Referencia a s, con la lambda también se puede indicar.

Esperar a que termine un hilo: **join**()

- Siempre se lanza un hilo principal (desde main) y a partir de este se van creando el resto de hilos.

- Para esperar a que un hilo termine se dispone del método **join**().

- Sólo se puede <u>llamar una vez</u> al método **join**.

- Se dispone de la función en thread: **joinable**() se aplica sobre un objeto thread y devuelve true / false para indicar si se puede hacer join a un hilo o no.

Vectores de hilos

- Los hilos se pueden combinar con la clase **vector** para tener varios hilos.

```
#include <vector>
#include <thread>

class Hilo {
	public:
	void operator()(){		// Muestra el identificador del hilo
		std::cout << "Dentro del hilo: " << std::this_thread::get_id() << " esta ejecutando" << std::endl;
	}
};

std::vector<std::thread> hilos;
// Creamos 10 hilos y se añaden al vector:
for (int i = 0 ; i < 10 ; i++)
	hilos.push_back(std::thread( (Hilo()) ));

// Ahora esperamos a que acaben todos los hilos:
std::cout << std::endl << "Esperamos por todos los hilos" << std::endl;
for (auto &h : hilos)
	h.join();
```

mutex

- Al igual que en POSIX los **mutex** (cerrojo) nos sirven para sincronizar el acceso de varios hilos a un recurso compartido para evitar condiciones de carrera y que se corrompa la memoria.

  - La 1ª forma: más propensa a errores se puede olvidar el desbloqueo del mutex:

```
#include <mutex>
miMutex.lock();      // Adquiere el cerrojo
// Actualizar el recurso;
miMutex.unlock();            // Libera el cerrojo
```

  - La 2ª forma: es más segura, se evita el posible error de la primera forma. El mutex se libera automáticamente.

```
std::lock_guard<std::mutex> guard(miMutex);
// Actualizar el recurso y después se libera automáticamente.
```

  - La 3ª forma: es equivalente a lock_guard → unique_lock
  - **unique_lock**<mutex> milock {miMutex};
  - // Actualizar el recurso y después se libera automáticamente.

lock_guard vs unique_lock

- **lock_guard** y **unique_lock** son más o menos lo mismo; lock_guard es una versión restringida con una interfaz limitada.

- **lock_guard** siempre tiene un candado desde su construcción hasta su destrucción.

- **unique_lock** puede crearse sin bloqueo inmediato, puede desbloquearse en cualquier momento de su existencia y puede transferir la propiedad del bloqueo de una instancia a otra.

- Por lo tanto, siempre utilizaremos lock_guard, a menos que se necesiten las capacidades de unique_lock.
- Una variable condition_variable necesita a unique_lock.

detach: Hilos no asociados

- Se puede indicar que un hilo sigue ejecutando después de que el destructor se ejecute con **detach()**.

- Útil para tareas que se ejecutan como demonios.

```
void actualiza () {
    for (;;) {
    muestra_reloj(stead_clock::now());
    this_thread :: sleep_for(second{1});
    }
}
void f () {
    thread t { actualiza };
    t .detach();
}
```

Problemas con hilos no asociados

- Inconvenientes:
  - Se pierde el control de qué hilos están activos.
  - No se sabe si se puede usar el resultado generado por un hilo.
  - No se sabe si un hilo ha liberado sus recursos.
  - Se podría acabar accediendo a objetos que han sido destruidos.

Variables de condición

- Mecanismo para sincronizar hilos en acceso a recursos compartidos:
  - wait(): Espera en un mutex.
  - notify_one(): Despierta a un hilo en espera.
  - notify_all(): Despierta a todos los hilos en espera.

- Productor / Consumidor
  - **class** peticion ;
  - queue<peticion> cola; // Cola de peticiones
  - condition_variable cv;
  - mutex m;
  - **void** productor();
  - **void** consumidor();

Consumidor

```
void consumidor() {
    for (;;) {
        unique_lock<mutex> l{m};
        while (cv.wait( l ) ) ;
        auto p = cola. front () ;
        cola.pop();
        l .unlock() ;
        procesa(p);
    };
}
```

- Efecto de **wait**
  - Libera el cerrojo y espera una notificación.
  - Adquiere el cerrojo al despertarse.

Productor

- **void** productor() {
  - **for** (;;) {
    - peticion p = genera();
    - unique_lock<mutex> l{m};
    - cola.push(p);
    - cv.notify_one() ;
  - }
- }

- Efecto de notify_one()
  - Despierta a uno de los hilos que están esperando en la condición.

Tareas asíncronas y future

- Una tarea **asíncrona** permite el lanzamiento simple de la ejecución de una tarea:
  - **En otro hilo** de ejecución.
  - Como una **tarea diferida**.

- Un **futuro** es un objeto que permite que un hilo pueda devolver un valor a la sección de código que lo invocó

Invocación de tareas asíncronas

```
#include <future>
#include <iostream>

int main() {
    std :: future<int> r = std :: async(tarea, 1, 10);
    otra_tarea() ;
    std :: cout << "Resultado= " << r.get() << std :: endl;
    return 0;
}
```

Uso de futuros

- **Idea general**:
  - Cuando un hilo necesita pasar un valor a otro hilo pone el valor en una **promesa**.
  - La implementación hace que el valor esté disponible en el correspondiente **futuro**.

- Acceso al **futuro** mediante **f.get()**:
  - Si se ha asignado un valor → obtiene el valor.
  - En otro caso → el hilo llamante se bloquea hasta que esté disponible.
  - Permite la transferencia transparente de excepciones entre hilos.

RAII, mutexes y lock guards

Gestión de memoria eficiente
malloc / free / RAII

- **RAII** → Resource Acquisition Is Initialization
  - Adquisición de recursos en la inicialización es el patrón más seguro y moderno para manejar memoria, archivos o sockets.
  - Evita fugas de memoria y asegura la liberación completa de memoria.
  - Como las versiones anteriores de C / C++
    - std::unique_ptr, std::vector, std::string en lugar de las funciones de bajo nivel de C, como malloc / calloc / realloc y free.
    - O los operadores de C++ new, new[], delete, delete[]

Smart Pointers

- **unique_ptr**: propiedad exclusiva del objeto.
- **make_unique**: crea un unique_ptr de forma segura.
- **shared_ptr**: propiedad compartida entre varios punteros.
- **make_shared**: crea un shared_ptr de forma eficiente.

Smart Pointers

- Se añadieron en **C++11** y representan un wrapper (envoltorio) de un puntero y gestionan su ciclo de vida.

- Tanto las funciones **malloc**, **calloc** y **realloc** de **C** y los operadores **new** y **delete** de **C++** son propensas para cometer errores de programación por no liberar la memoria adecuadamente. Para paliar estos problemas se añadieron los Smart pointers.

- Se encuentran en el fichero de cabecera: **memory**
  - #include <memory>
  - Básicamente tenemos 2 tipos: **unique_ptr** y **shared_ptr**

Smart Pointers

- Se definen con tipos parametrizados en los cuales se puede utilizar tipos básicos de C, clases o tipos creados por el programador. Ya sean clases o estructuras.

- Los Smart pointers, a parte, de encargarse de la reserva de memoria se pueden comportar como los punteros raw. Añadiendo:
  - Se pueden establecer a **nullptr**.
  - Se puede utilizar: ***ptr** para desreferenciar ptr.
  - Operador flecha para acceder a los métodos: **ptr->** método()

- Adicionalmente añaden:
  - **ptr.get**():
    - Retorna el puntero raw que gestiona por debajo.
  - **ptr.reset(raw_ptr)**
    - Para la gestión del puntero, libera la memoria ocupada y establece ptr a ptr_raw.

Unique pointer (std::unique_ptr)

- Sirve para construir un único puntero que libera automáticamente la memoria ocupada por este cuando deja de utilizarse.
- Suponiendo un tipo cualquiera llamado: **Type**

```
#include <memory>
// Utilizando el constructor por defecto de Type (se puede inferir el tipo con auto).
    auto p = std:: unique_ptr <Type >(new Type);

// Utilizando el constructor con parámetros de Type:
    auto p = std:: unique_ptr <Type >(new Type(<params >));

// También se puede utilizar la función: make_unique
    auto p = std::make_unique<Type>(<params>)
```

Porque es único

- No tiene constructor copia.
- No puede ser copiado.
- Se puede mover.
- Garantiza que la memoria sea siempre propiedad de un único puntero.

Ejemplo

```
#include <iostream >
#include <memory >

struct A {
    int a = 10;
};

int main() {
    auto a_ptr = std:: unique_ptr <A>(new A);
    std::cout << a_ptr ->a << std:: endl;
    auto b_ptr = std::move(a_ptr);
    std::cout << b_ptr ->a << std:: endl;
    return 0;
}
```

Consecuencias

- Pasar por parámetro un unique_ptr a una función trae consecuencias:
  - No se puede copiar, al pasarlo por parámetro se tiene que realizar una copia del puntero (y está eliminado el constructor copia).
  - Se tiene que mantener como único.

- Si necesitamos pasarlo como parámetro se puede hacer uso de la función **std::move** (mueve la memoria del puntero).

Ejemplos

```
void funcion(std::unique_ptr<Clase> arg){
    //Contenido de la función
}

int main(){
    std::unique_ptr<Clase> instancia {new clase};
    // Da error:
    funcion(instancia);

    // Podemos utilizar:
    funcion(std::move(instancia));
}
```

Shared pointer (std::shared_ptr)

- Dispone de un constructor como unique_ptr
- Se puede copiar.
- Almacena un contador interno y un puntero raw.
  - Incrementa el contador en 1, cuando se copia.
  - Decrementa el contador en 1 cuando se destruye.
- Libera la memoria cuando el contador es 0.
- Se pueden inicializar con un unique_ptr.

Ejemplo

```
#include <memory >

// Utilizando el constructor por defecto: Type();
auto p = std:: shared_ptr <Type >(new Type);
auto p = std:: make_shared <Type >();

// Utilizando el constructor con parámetros: Type(<params >);
auto p = std:: shared_ptr <Type >(new Type(<params >));
auto p = std:: make_shared <Type >(<params >);
```

Ejemplo

```
#include <iostream >
#include <memory >

struct A {
        A(int a) { std::cout << “Se crea el objeto...\n"; }
        ~A() { std::cout << “Se elimina el objeto...\n"; }
};

int main() {
    // Equivalente a: std:: shared_ptr <A>(new A(10));
    auto a_ptr = std:: make_shared <A >(10);
    std::cout << a_ptr.use_count () << std::endl;
    {
        auto b_ptr = a_ptr;
        std::cout << a_ptr.use_count () << std::endl;
    }
    std::cout << “En el scope de main\n";
    std::cout << a_ptr.use_count () << std::endl;
    return 0;
}
```

Cuando utilizarlos

- Utilizar punteros inteligentes para administrar la memoria dinámica.
- Por defecto, utilizar **unique_ptr**.
- Si hay varios objetos que deben compartir la propiedad, utilizar **shared_ptr**.
- El uso de punteros inteligentes evita tener destructores en las clases.
- Se aconseja utilizar las funciones make_unique y make_shared, si no, inicializar con un puntero raw.

Ejemplo

- En el caso de llamar a una función un puntero **shared_ptr** se puede copiar el puntero (por debajo se llama al constructor copia y contador interno del puntero se incrementa en uno)

```
void funcion2(std::shared_ptr<Alertador> arg){
    std::cout << "Se ejecuta la función 2: count: " << arg.use_count() << "\n";

}

int main(){
    std::shared_ptr<Alertador> ps {new Alertador};
    funcion2(ps);
}
```

Situaciones

- Cuando utilizamos un **shared_ptr** en una serie de ámbitos anidados y que apuntan a la misma instancia (el contador interno del puntero cada vez que termina un ámbito se libera un puntero (y se descuenta el contador)) Cuando el contador llega a 0 se libera el recurso y se llama al destructor.

Errores habituales

```
#include <iostream >
#include <memory >
int main() {
    int a = 0;
    // Hay que inicializar con memoria del heap.
    auto a_ptr = std:: unique_ptr <int >(&a);
    // Se producirá un error al intentar liberar.
    return 0;
}
```

Ejemplo

```
#include <iostream >
#include <vector >
#include <memory >
using std::cout; using std:: unique_ptr;
struct AbstractShape { // Structs to save space.
    virtual void Print () const = 0;
};
struct Square : public AbstractShape {
    void Print () const override { cout << "Square\n"; }
};
struct Triangle : public AbstractShape {
    void Print () const override { cout << "Triangle\n"; }
};
int main() {
    std::vector <unique_ptr <AbstractShape >> shapes;
    shapes.emplace_back(new Square);
    auto triangle = unique_ptr <Triangle >(new Triangle);
    shapes.emplace_back(std::move(triangle));
    for (const auto& shape : shapes) { shape ->Print (); }
    return 0;
}
```

Ejemplo para arrays

Para compilar: -std=c++14

```
unique_ptr array = make_unique<int []>(N);
```

N indica el número de posiciones del Array.

```
for (int i = 0 ; i < N ; i++)
        array[i] = i+1;

for (int i = 0 ; i < N ; i++)
        cout << array[i] << ", ";
cout << endl;
```

Clase Punto

```
class Punto {

        friend ostream & operator<<(ostream &os, const Punto &p){
                return os << "[x = " << p.x << ", y = " << p.y << "]";
        }
        int x, y;

        public:
                Punto(int _x=0,int _y=0):x(_x),y(_y){}
                int getX(){ return this->x; }
                int getY(){ return this->y; }
};
```

Ejemplo para arrays

```
void test_array_Puntos(){
        cout << endl << "TEST ARRAY DE PUNTOS:" << endl;
        unique_ptr array = make_unique<Punto []>(N);

        for (int i = 0 ; i < N ; i++)
                cout << array[i] << endl;
        cout << endl;
}
```

Ejemplo: Con un objeto

```
void test_ptr_Punto(){
        cout << endl << "TEST PUNTERO A OBJETO PUNTO:" << endl;
        unique_ptr<Punto> ppunto = make_unique<Punto>(5,-8);
        cout << "X: " << ppunto->getX() << endl;
        cout << "Y: " << ppunto->getY() << endl;

        cout << "Punto: " << *ppunto << endl;
}
```

reset

- Se puede llamar al método **reset** y se va a liberar el **shared_ptr**.

```
int main(){
    std::shared_ptr<Clase> instancia {new Clase};
    instancia.reset(); → Salta el destructor!
    return 0;
}
```

make_shared

- Hay una función **<u>más eficiente</u>** que es **make_shared**.
  - std::shared_ptr<Clase> instancia = std::make_shared<Clase>();

- Tanto con shared_ptr como con unique_ptr se puede utilizar auto en las declaraciones:
  - auto ps3 = std::make_shared<Clase>();

Notas

- **No se aconseja utilizar unique_ptr para crear una matriz**
- Y make_shared<int []> No está soportado. Incluso ni en C++17
- Se aconseja utilizar **vector<vector<int>>** o la clase **array** de C++

Minimización de la latencia en sistemas distribuidos

- **Serialización rápida**
  - nlohmann/json es cómodo de usar y fácil pero no es de las opciones más rápidas para grandes volúmenes.
  - Disponemos de formatos binarios como MessagePack, CBOR o FlatBuffers.

Minimización de la latencia en sistemas distribuidos

- **Comunicación eficiente:**
  - Utilizar **HTTP/2** o **gRPC**
  - Minimizar el tamaño de las respuestas y evitar redirecciones.

- **Concurrencia y paralelismo**
  - Crow permite utilización de varios: con .**multithreaded**() pero las estructuras que estén compartidas tienen que estar protegidas.

Minimización de la latencia en sistemas distribuidos

- **Caching inteligente**:
  - Cachear los resultados de las consultas frecuentes en memoria (std::unordered_map).
  - Evitar llamadas repetidas a base de datos o servicios externos.

- **Lazy loading** y procesamiento diferido:
  - Procesar solo lo necesario en cada solicitud

Ejemplo de caché en memoria

- Una posible colección para la implementación puede ser: **std::unordered_map** protegida con un **mutex**.

- #include <unordered_map>
- #include <string>
- #include <mutex>

- std::unordered_map<std::string, std::string> cache;
- std::mutex cache_mutex;

Ejemplo de caché en memoria

```
std::string get_from_cache(const std::string& key) {
   std::lock_guard<std::mutex> lock(cache_mutex);
   auto it = cache.find(key);
   return (it != cache.end()) ? it->second : "";
}

void set_in_cache(const std::string& key, const std::string& value) {
   std::lock_guard<std::mutex> lock(cache_mutex);
   cache[key] = value;
}
```

Ejemplo de caché en memoria

```
CROW_ROUTE(app, "/datos/<string>")
([](const crow::request& req, const std::string& clave){
  std::string resultado = get_from_cache(clave);
  if (!resultado.empty()) {
    CROW_LOG_INFO << "Respuesta desde caché";
    return crow::response(200, resultado);
  }

  // Simular cálculo o consulta costosa
  std::string nuevo_resultado = "Resultado para " + clave;

  set_in_cache(clave, nuevo_resultado);
  return crow::response(200, nuevo_resultado);
});
```

Consideraciones

- Implementar una clase con una plantilla que nos sirva la caché para distintos tipos de objetos.

```
template<typename T>
class MemCache {
    private:
        std::unordered_map<std::string, T> data_;
        std::mutex mutex_; // Para el acceso concurrente a la colección
}
```

Comparativa
std::thread / Boost.Asio

std::thread / Boost.Asio

- **std::thread** es la clase estándar de C++ para crear y manejar hilos. Te permite ejecutar funciones en paralelo.
  - Necesitas manejar sincronización con std::mutex, std::condition_variable, etc.
  - No escala bien para miles de conexiones simultáneas (como en servidores web).

- **Boost.Asio** es una librería para **programación asíncrona y basada en eventos**, ideal para manejar múltiples conexiones de red sin bloquear hilos.
  - Manejo eficiente de miles de conexiones con pocos hilos.
  - Compatible con std::thread, std::future.
  - Ideal para microservicios, servidores HTTP, y sistemas embebidos.

Comparativa

Herramienta	Ideal para...	Evítalo si...
`std::thread`	Tareas paralelas simples, procesamiento	Necesitas escalabilidad o IO intensiva
`Boost.Asio`	Servidores concurrentes, IO no bloqueante	Tu aplicación es muy simple o CPU-bound

CPU-bound:
El programa **consume mucho tiempo de CPU** realizando cálculos intensivos.
El cuello de botella está en la **velocidad de procesamiento**, no en la espera por datos externos.
Aumentar el número de núcleos o la frecuencia del procesador puede mejorar el rendimiento.

Boost.Asio

¿Qué es Boost.Asio?

- **Boost.Asio** es una librería de C++ para **programación asíncrona y basada en eventos**, especialmente útil para:
  - Redes TCP/UDP
  - Timers
  - Serialización
  - **Multithreading**
  - I/O no bloqueante
- Está diseñada para construir aplicaciones **eficientes, escalables y concurrentes**, como servidores web, microservicios, sistemas embebidos o clientes de red.

Características

- **Modelo asíncrono**: evita bloqueos usando callbacks.
- **Sin dependencias externas**: todo se basa en C++ estándar y Boost.
- **Multiplataforma**: funciona en Windows, Linux, macOS.
- **Integración con** std::thread **y** std::future para concurrencia moderna.
- **Timers y señales**: ideal para tareas periódicas o eventos del sistema.

Ejemplo

```
boost::asio::io_context io;
tcp::acceptor acceptor(io, tcp::endpoint(tcp::v4(), 1234));

while (true) {
    tcp::socket socket(io);
    acceptor.accept(socket);
    // Manejar la conexión
}
```

Uso de Boost.Asio

Sector	Aplicación típica
Microservicios	Servidores HTTP, gRPC, ZeroMQ integrados
Sistemas embebidos	Comunicación entre sensores y controladores
Telecomunicaciones	Procesamiento de paquetes y señalización
Juegos en red	Sincronización de estado y eventos

Cuando elegir Boost.Asio

- Manejar miles de conexiones simultáneas sin bloquear hilos.
- Trabajar en sistemas de alto rendimiento o embebidos.
- Solución ligera y sin dependencias externas como gRPC o RabbitMQ

Multicliente con boost.asio

- Tenemos que utilizar
  - **async_read** y **async_write**
  - Para no bloquear el flujo del servidor.

  - El servidor escucha por un puerto:
  - Acepta múltiple conexiones TCP
  - Leer solicitudes HTTP
  - Responder un mensaje
  - Cierra la conexión después de enviar la respuesta.

- **Flujo de ejecución:**
- **main**():
  - Crea el io_context, el endpoint y el acceptor.
  - Llama a do_accept() para comenzar a aceptar conexiones.
  - Ejecuta el bucle de eventos con ioc.run().

- **do_accept**():
  - Espera una conexión entrante.
  - Cuando llega, crea una nueva session y llama a start().

- **session::start**():
  - Inicia la lectura de la solicitud HTTP con http::async_read.

- **session::handle_request**():
  - Procesa la solicitud y prepara una respuesta HTTP

- **session::write_response**():
  - Envía la respuesta al cliente y cierra el socket.

Objetos del Servidor

Objeto / Clase	Descripción
`asio::io_context`	Motor de eventos de Boost.Asio. Coordina operaciones asíncronas.
`tcp::endpoint`	Representa la dirección IP y el puerto donde el servidor escucha (en este caso, `0.0.0.0:8080`).
`tcp::acceptor`	Acepta conexiones entrantes de clientes TCP.
`tcp::socket`	Canal de comunicación con el cliente. Se usa para leer y escribir datos.
`session`	Clase que maneja la comunicación con un cliente individual. Se crea una instancia por conexión.
`beast::flat_buffer`	Buffer de Boost.Beast para almacenar datos leídos del socket.
`http::request<http::string_body>`	Representa la solicitud HTTP recibida del cliente.
`http::response<http::string_body>`	Representa la respuesta HTTP que se enviará al cliente.

Multicliente

- La clave está en cómo se manejan las conexiones:
  - Usa tcp::acceptor::async_accept() para aceptar conexiones **de forma asíncrona**.
  - Cada vez que se acepta una conexión, se crea una nueva instancia de **session** con **std::make_shared<session>(...).**
  - Dentro de **do_accept**(), se llama recursivamente a sí mismo para seguir aceptando más clientes:
    - do_accept(acceptor); // vuelve a aceptar otra conexión
  - Cada session maneja su propia conexión de forma independiente usando **shared_from_this**().

session::start()

- **No son un hilo en sí**, pero **funcionan como una unidad de trabajo asíncrona**, muy parecida a lo que haría un hilo en un servidor multicliente.
  - ***Se crea una sesión por cada conexión entrante***

- Usa operaciones **asíncronas** (async_read, async_write) que se ejecutan dentro del asio::io_context.

- No crea un hilo nuevo, pero se comporta como una tarea independiente que puede ejecutarse en paralelo con otras sesiones si el **io_context** tiene varios hilos activos.

session

- Cada cliente tiene su propia session, con su propio socket, buffer, solicitud y respuesta.

- Esto permite que múltiples clientes se atiendan en paralelo, sin interferencias entre ellos.

- Como las operaciones son asíncronas, no necesitas un hilo por cliente, lo que permite escalar a miles de conexiones

Dentro de la session

- Representa una sesión HTTP individual.
  - Leer solicitud HTTP
  - Procesarla
  - Enviar respuesta
  - Cerrar la conexión

Varios threads

- **Se pueden usar varios núcleos del procesador**:

```
std::vector<std::thread> threads;
for (int i = 0; i < std::thread::hardware_concurrency(); ++i) {
  // Construye el thread dentro del vector para evitar copias
   threads.emplace_back([&ioc] { ioc.run(); });
}
for (auto& t : threads) t.join();
```